Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Busca em Grafos

2

Busca em Grafos

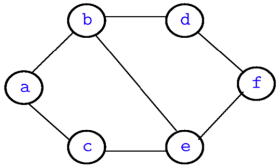
Problema fundamental em grafos:

Como explorar um grafo de forma sistemática?

- Muitas aplicações são abstraídas como problemas de busca
- Os algoritmos de busca em grafos são a base de vários algoritmos mais gerais em grafos

3

Busca em Grafos



- Como explorar o grafo?
- Por exemplo, como saber se existe caminhos simples entre dois vértices?
- Evitar explorar vértices já explorados. Temos que marcar os vértices!

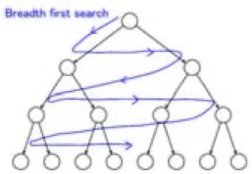
4

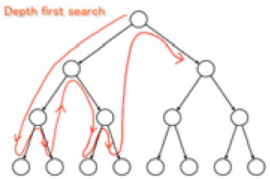
Busca em grafos

Objetivo: sistematicamente explorar todos os vértices e arestas de um grafo.

Dois tipos de busca:

- *Busca em largura (Breadth First Search – BFS). –Busca em amplitude (Breadth First Search – BFS).*
- *Busca em profundidade (Depth First Search – –Busca em profundidade (Depth First Search – DFS).*





5

Busca em largura (BFS)

- Representa um dos mais simples algoritmos de busca em grafos.
- É usado como modelo para alguns algoritmos importantes:
  - Menor caminho;
  - Árvore de cobertura mínima;
- Similar ao caminhamento por níveis em árvores.
- Idéia: processa os vértices por níveis, começando por aqueles vértices mais próximos do vértice inicial ***s*** e deixando os vértices mais próximos do vértice inicial *s* e deixando os vértices mais distantes para depois.

6

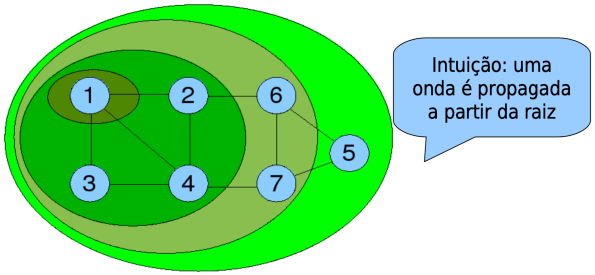
Busca em largura (BFS)

- Seja G = (V , A) e um vértice s de G
- BFS percorre as arestas de G descobrindo todos os vértices atingíveis a partir de s.
- BFS determina a distância (em número de arestas) de cada um desses vértices a s.

7

Busca em largura (BFS)

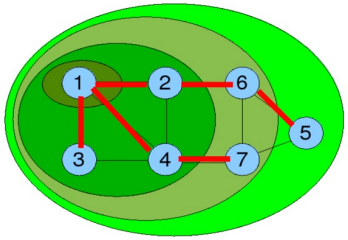
- Antes de encontrar um vértice à distância k+1 de s, todos os vértices à distância k são encontrados.



8

Busca em largura (BFS)

- BFS produz uma árvore BFS com raiz em s, que contém todos os vértices acessíveis. Determinando o caminho mínimo ou caminho mais curto (caminho que contém o número mínimo de arestas) de s a t (vértice acessível).



9

Busca em largura (BFS)

- Para organizar o processo de busca os vértices são pintados:
  - branco: não foram descobertos ainda
  - cinzas: são a fronteira. O vértice já foi descoberto mais ainda não examinamos os seus vizinhos.
  - pretos: são os vértices já descobertos e seus vizinhos já foram examinados.

- É utilizada uma fila para manter os vértices cinzas

10

```
BFS(V, A, s)
for each u in V – {s}
        color[u] ← WHITE
        d[u] ← infinity
        π[u] ← NIL
color[s] ← GRAY
d[s]←0
π[s] ← NIL
Q←{}
ENQUEUE(Q, s)
while Q is non-empty
        u ← DEQUEUE(Q)
        for each v adjacent to u
                if color[v] = WHITE
                        d[v] ← d[u] + 1
                        π[v] ← u
                        ENQUEUE(Q, v)
color[u] ← BLACK
```

- Para cada vértice V exceto s
- No início, todos os vértices são **brancos**

11

```
BFS(V, A, s)
for each u in V – {s}
        color[u] ← WHITE
        d[u] ← infinity
        π[u] ← NIL
color[s] ← GRAY
d[s]←0
π[s] ← NIL
Q←{}
ENQUEUE(Q, s)
while Q is non-empty
        u ← DEQUEUE(Q)
        for each v adjacent to u
                if color[v] = WHITE
                        d[v] ← d[u] + 1
                        π[v] ← u
                        ENQUEUE(Q, v)
color[u] ← BLACK
```

- Vértice de origem descoberto
- **Inicializa as variáveis auxiliares da origem s e a fila Q**

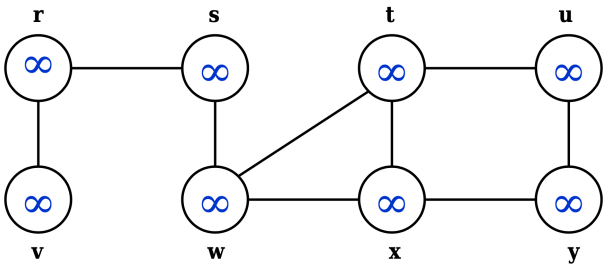
12

```
BFS(V, A, s)
for each u in V – {s}
        color[u] ← WHITE
        d[u] ← infinity
        π[u] ← NIL
color[s] ← GRAY
d[s]←0
π[s] ← NIL
Q←{}
ENQUEUE(Q, s)
while Q is non-empty
        u ← DEQUEUE(Q)
        for each v adjacent to u
                if color[v] = WHITE
                        d[v] ← d[u] + 1
                        π[v] ← u
                        ENQUEUE(Q, v)
color[u] ← BLACK
```

- Colocar o vértice origem na fila Q
- Enquanto existam vértices cinzas
- Para cada vértice adjacente a u
- Se é branco ele ainda não foi descoberto
- Pai de vé o nó que levou a descoberta de v
- Os vizinhos de u já foram examinados

13

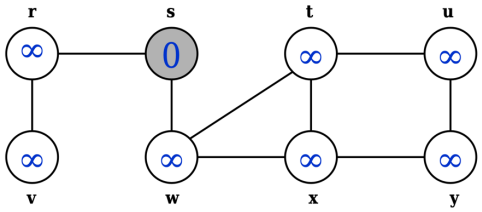
Busca em largura (BFS)



No início todos os vértices são brancos e a distância é infinita!

14

Busca em largura (BFS)

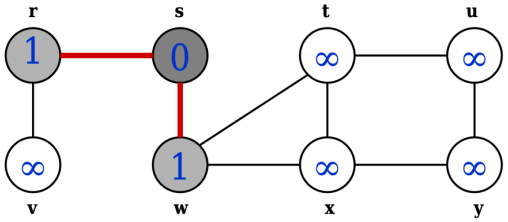


O vértice origem é pintado de cinza (ele é considerado descoberto) e é colocado na fila

Q: | s |

15

Busca em largura (BFS)

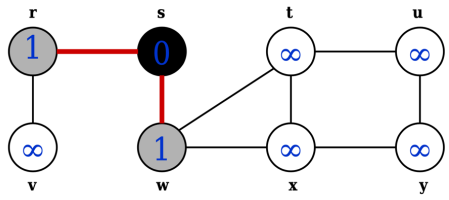


É retirado o primeiro elemento da fila (s) e os adjacentes a ele são colocados em Q, pintados de cinza. Além disso é atualizada a distância e o pai

Q: | w | r |

16

Busca em largura (BFS)

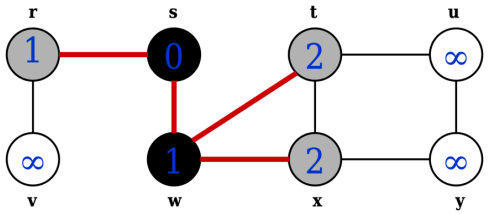


Colorimos o vértice com preto (os seus vizinhos já foram descobertos)

Q: | w | r |

17

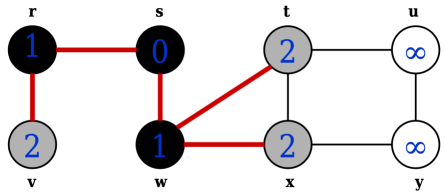
Busca em largura (BFS)



Q: | r | t | x |

18

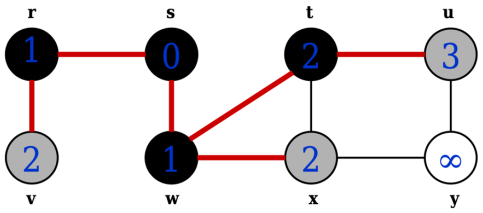
Busca em largura (BFS)



Q: | t | x | v |

19

Busca em largura (BFS)

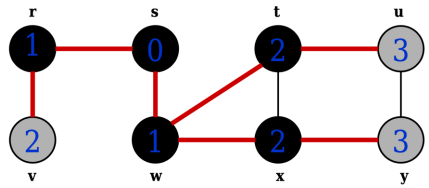


Observe que somente enfileiramos vértices brancos, que são imediatamente coloridos de cinza ao entrar na fila

Q: | x | v | u |

20

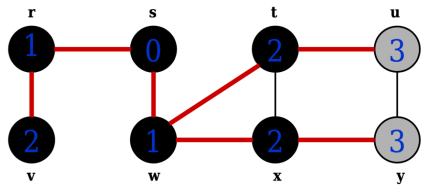
Busca em largura (BFS)



Q: | v | u | y |

21

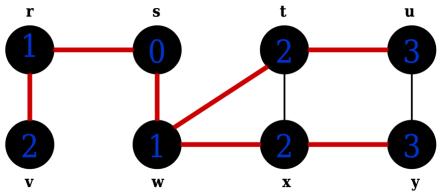
Busca em largura (BFS)



Q: | u | y |

22

Busca em largura (BFS)



Q: Ø

23

Caminho entre dois vértices

- Problema: Como saber se existe caminho entre dois vértices?
- Exemplo: existe um caminho entre Londrina e Manaus?
  - ✓ Solução: usar BFS;
  - ✓ Marcar Londrina como raiz;
  - ✓ Realizar BFS
  - ✓ Ao terminar BFS se Manaus tiver distância diferente de infinito ou se o vértice está pintado de preto, então há caminho, caso contrário, não há

24

Grafo conectado

Problema: Como saber se um grafo é conectado (i.e., se cada par de vértices está conectado por um caminho)?

- Exemplo: gostaria de saber se posso voar de qualquer cidade para qualquer cidade.

- Solução: usar BFS
- Escolher um vértice v qualquer de G
- Executar BFS à partir de v
- Verificar se todos vértices foram pintados de preto

25

Grafo conectado

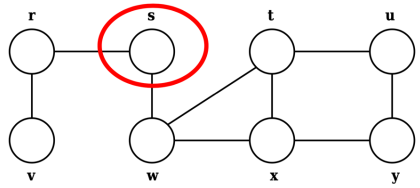
Problema: Como saber se um grafo é conectado (i.e., se cada par de vértices está conectado por um caminho)?

- Exemplo: gostaria de saber se posso voar de qualquer cidade para qualquer cidade.

```
R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
        R = R U {v}
```

26

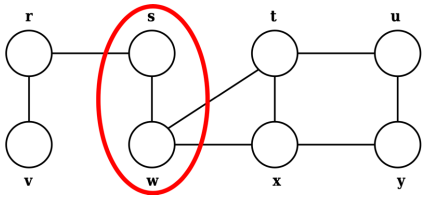
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

27

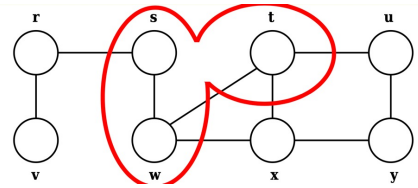
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

28

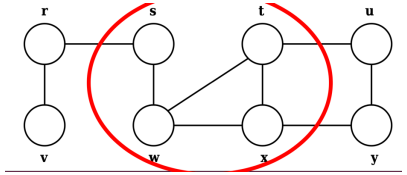
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

29

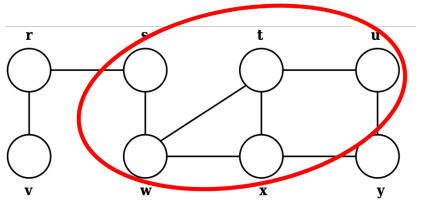
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

30

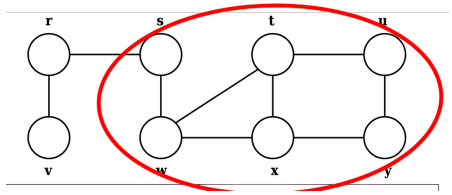
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

31

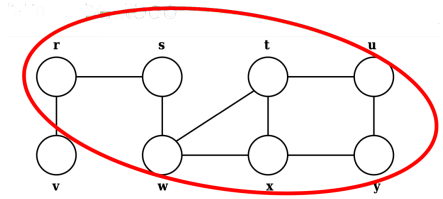
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

32

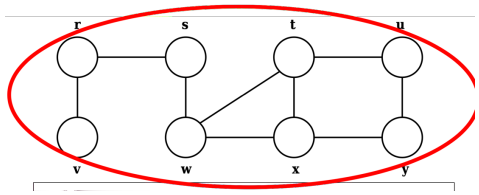
Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

33

Grafo conectado



R = {s}
while existir aresta (u,v) em que u pertence a R e v não pertence a R
R = R ∪ {v}

- Se R contém todos os vértices, então o grafo é conectado, caso contrário, não é.

34

Busca em profundidade (DFS)

- É outro método de busca
- A idéia é proseguir a busca sempre a partir do vértice descoberto mais recentemente, até que este não tenha mais vizinhos descobertos. Neste caso, volta-se na busca para o precursor desse vértice.
- Oposto de BFS que explora o vértice mais antigo primeiro
- DFS devolve uma floresta

35

Busca em profundidade (DFS)

- A idéia principal é buscar verticalmente, sempre que possível.
- Sempre que um novo vértice *v* é descoberto, ele deve ser explorado por completo.
- Quando *v* é totalmente explorado, fazer backtracking totalmente explorado, fazer backtracking para o seu predecessor.

36

Busca em profundidade (DFS)

Intuição: Procurar uma saída de um labirinto

- vai fundo atrás da saída (tomando decisões a cada encruzilhada)
- volta a última encruzilhada quando encontrar um beco sem saída ou encontrar um lugar já visitado.

37

Busca em profundidade (DFS)

Os vértices recebem 2 rótulos:

- d[.]: o momento em que o vértice foi descoberto (tornou-se cinza)
- f[.]: o momento em que examinamos os seus vizinhos (tornou-se preto)
- O vértice é branco até d , cinza entre d e f e preto a partir de f.

38

```
DFS (V, A)
for each vertex u in V
        color[u] ← WHITE
        π[u] ← NIL
time←0
for each vertex u in V
        if color[u] = WHITE
        then DFS-Visit(u)
DFS-Visit(u)
color[u] ← GRAY
time←time+1
d[u] ← time
for each vertex v adjacent to u
        if color[v] = WHITE
        then π[v] ← u
                DFS-Visit(v)
color[u] ← BLACK
time ← time + 1
f[u] ← time
```

Inicialização

39

```
DFS (V, A)
for each vertex u in V
        color[u] ← WHITE
        π[u] ← NIL
time←0
for each vertex u in V
        if color[u] = WHITE
        then DFS-Visit(u)
DFS-Visit(u)
color[u] ← GRAY
time←time+1
d[u] ← time
for each vertex v adjacent to u
        if color[v] = WHITE
        then π[v] ← u
                DFS-Visit(v)
color[u] ← BLACK
time ← time + 1
f[u] ← time
```

Cria a floresta (cria uma nova árvore a partir de u)

40

```
DFS (V, A)
for each vertex u in V
        color[u] ← WHITE
        π[u] ← NIL
time←0
for each vertex u in V
        if color[u] = WHITE
        then DFS-Visit(u)
DFS-Visit(u)
color[u] ← GRAY
time←time+1
d[u] ← time
for each vertex v adjacent to u
        if color[v] = WHITE
        then π[v] ← u
                DFS-Visit(v)
color[u] ← BLACK
time ← time + 1
f[u] ← time
```

Vértice u acabou de ser descoberto

41

```
DFS (V, A)
for each vertex u in V
        color[u] ← WHITE
        π[u] ← NIL
time←0
for each vertex u in V
        if color[u] = WHITE
        then DFS-Visit(u)
DFS-Visit(u)
color[u] ← GRAY
time←time+1
d[u] ← time
for each vertex v adjacent to u
        if color[v] = WHITE
        then π[v] ← u
                DFS-Visit(v)
color[u] ← BLACK
time ← time + 1
f[u] ← time
```

Explora a aresta (u, v)

42

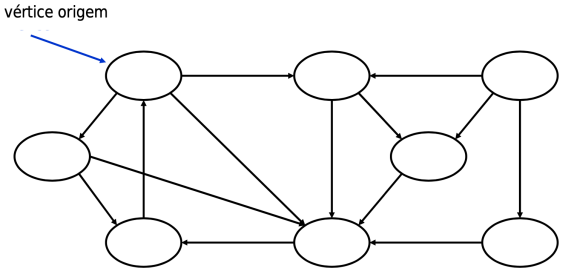
```
DFS (V, A)
for each vertex u in V
        color[u] ← WHITE
        π[u] ← NIL
time←0
for each vertex u in V
        if color[u] = WHITE
        then DFS-Visit(u)
DFS-Visit(u)
color[u] ← GRAY
time←time+1
d[u] ← time
for each vertex v adjacent to u
        if color[v] = WHITE
        then π[v] ← u
                DFS-Visit(v)
color[u] ← BLACK
time ← time + 1
f[u] ← time
```

Os vizinhos de u já foram examinados

Terminamos com o vértice u

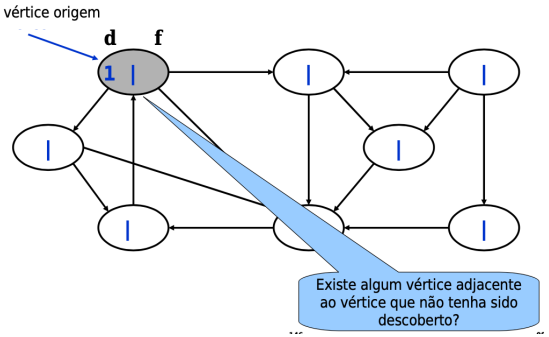
43

Busca em profundidade (DFS)



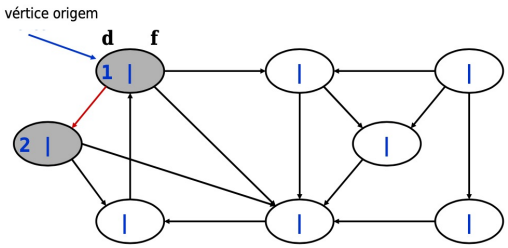
44

Busca em profundidade (DFS)



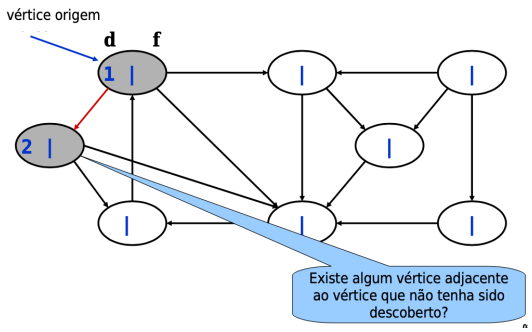
45

Busca em profundidade (DFS)



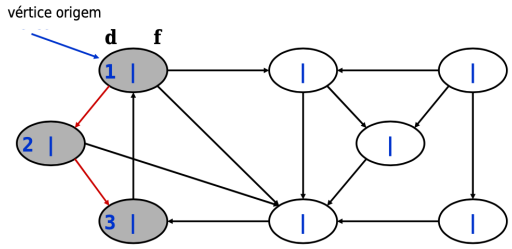
46

Busca em profundidade (DFS)



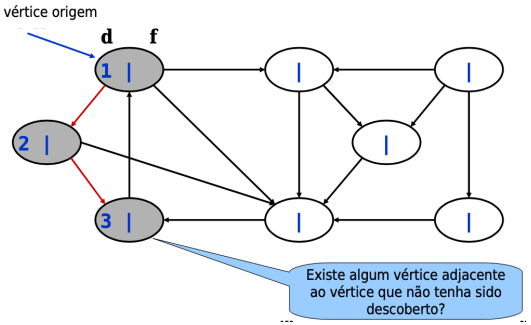
47

Busca em profundidade (DFS)



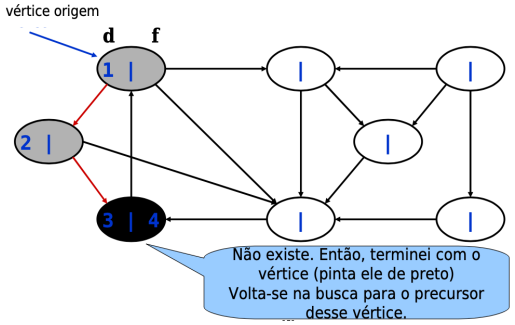
48

Busca em profundidade (DFS)



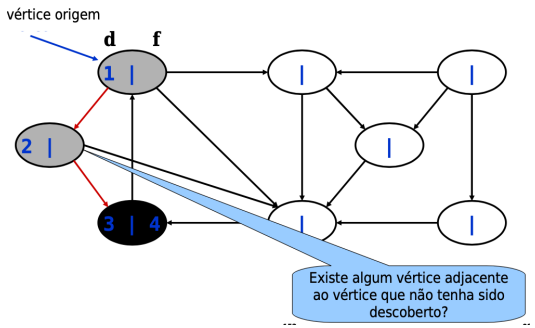
49

Busca em profundidade (DFS)



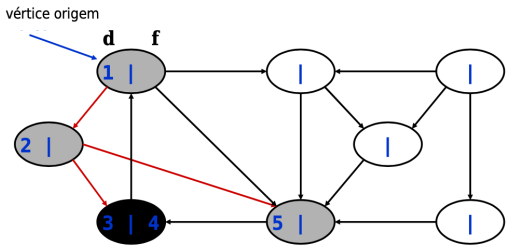
50

Busca em profundidade (DFS)



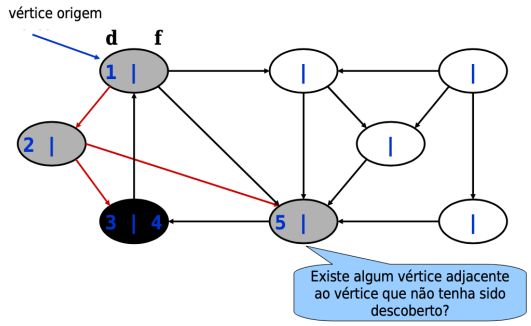
51

Busca em profundidade (DFS)



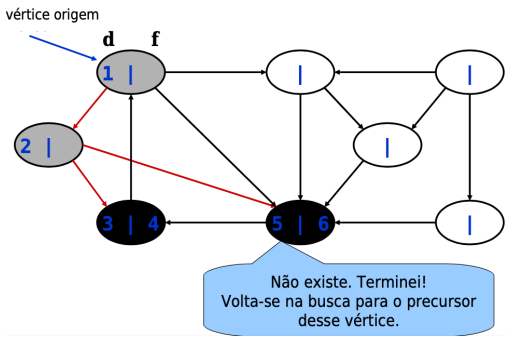
52

Busca em profundidade (DFS)



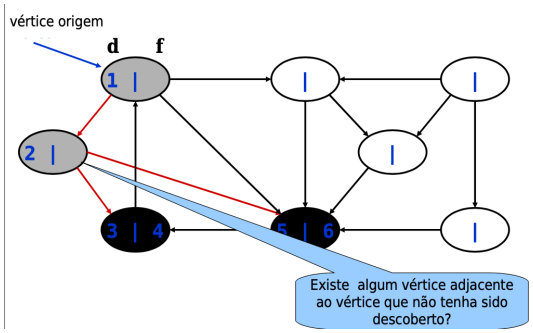
53

Busca em profundidade (DFS)



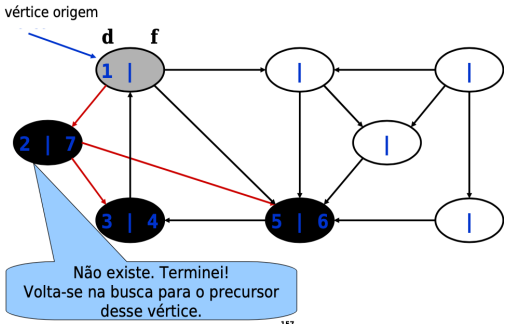
54

Busca em profundidade (DFS)



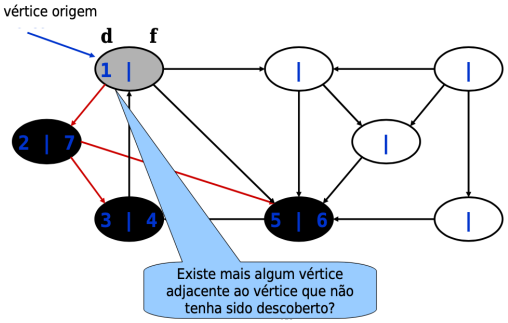
55

Busca em profundidade (DFS)



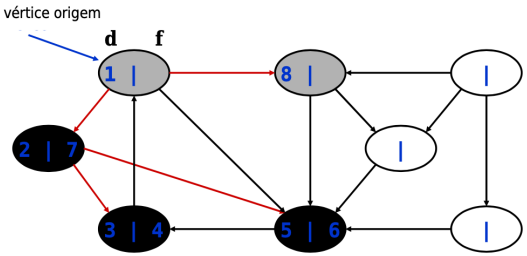
56

Busca em profundidade (DFS)



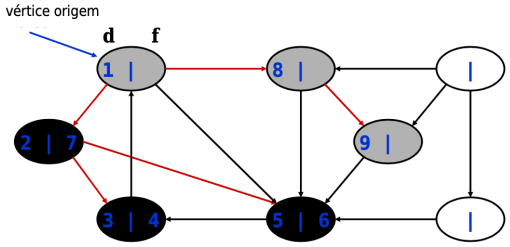
57

Busca em profundidade (DFS)



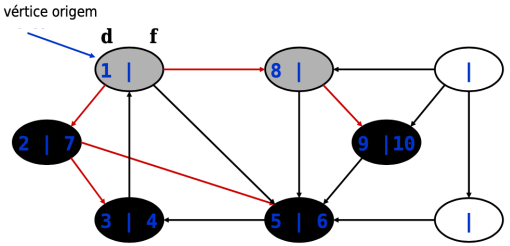
58

Busca em profundidade (DFS)



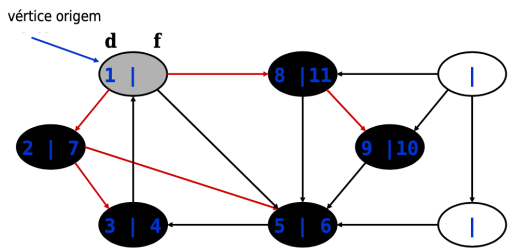
59

Busca em profundidade (DFS)



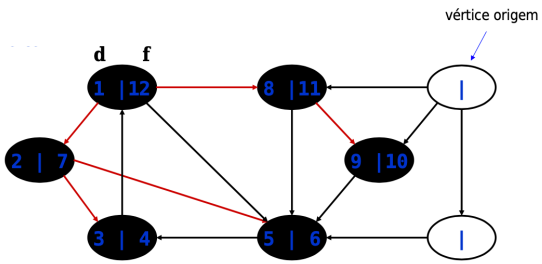
60

Busca em profundidade (DFS)



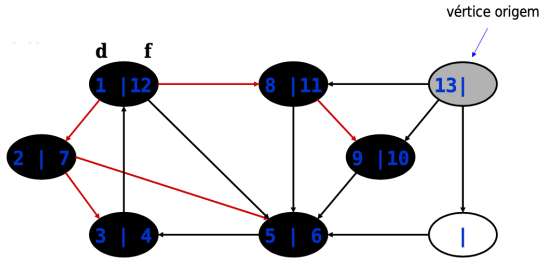
61

Busca em profundidade (DFS)



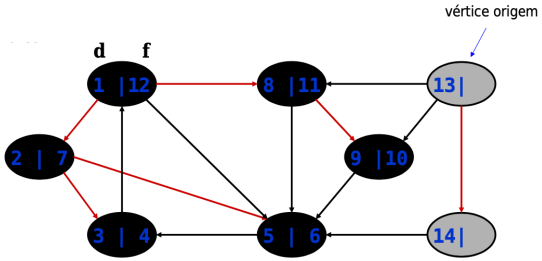
62

Busca em profundidade (DFS)



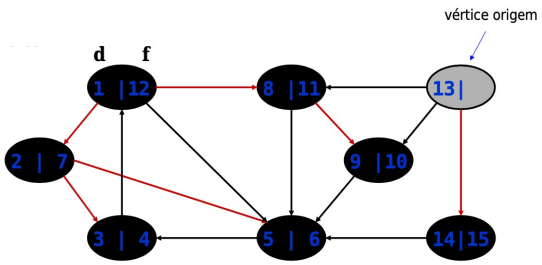
63

Busca em profundidade (DFS)



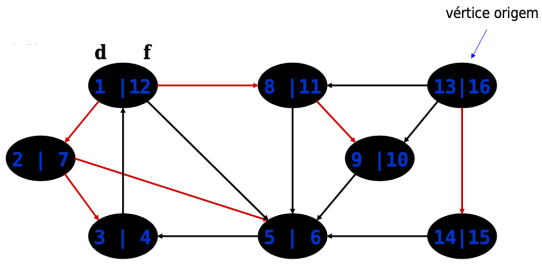
64

Busca em profundidade (DFS)



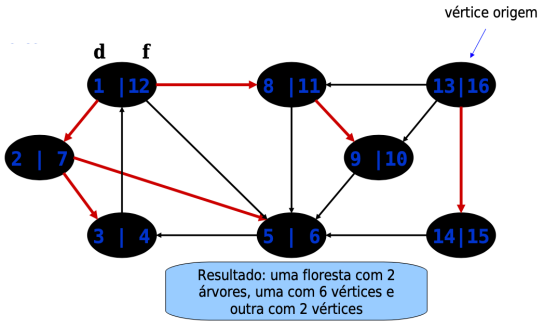
65

Busca em profundidade (DFS)



66

Busca em profundidade (DFS)



67

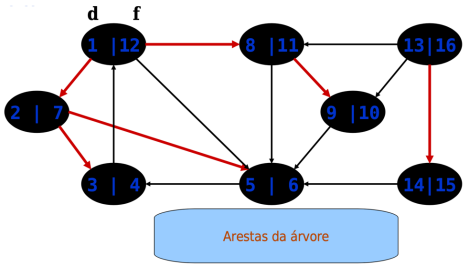
Busca em profundidade – Classificação das arestas

- Arestas da árvore
- Arestas de retorno
- Arestas de avanço
- Arestas de cruzamento

68

Busca em profundidade – Classificação das arestas

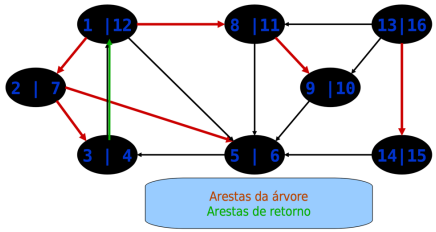
- Arestas da árvore: são arestas que pertencem a alguma das árvores DFS da floresta.



69

Busca em profundidade – Classificação das arestas

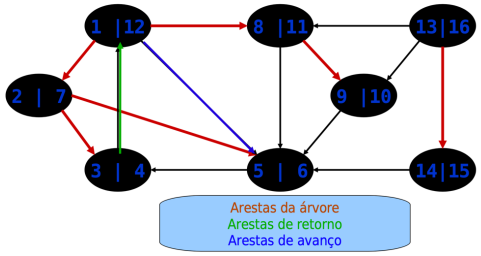
- Arestas de retorno: arestas (u,v) que não pertencem a árvore DFS. Conectam um vértice u com um ancestral v em uma árvore DFS. Self- loops são considerados arestas de retorno.



70

Busca em profundidade – Classificação das arestas

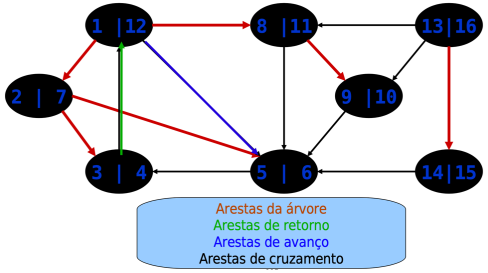
- Arestas de avanço: arestas (u,v) que não pertencem a árvore DFS. Conectam um vértice u a um descendente v em uma árvore DFS.



71

Busca em profundidade – Classificação das arestas

- Arestas de cruzamento: Todas as outras arestas.



72